

Cahier des charges

Master Cyber – Chevalier Enzo

Table des matières

Contexte et objectif	3
Description du projet	3
Objectif du projet	3
Analyse de l'existant	4
Wix	4
Squarespace	4
Webflow	5
Portfoliobox	5
Synthèse	5
Cahier des charges	6
Exigences fonctionnelles	6
Exigences non fonctionnelles	10
Méthodologie & Organisation du travail	12
Méthodologie	12
Organisation du travail	12
Planning (Gantt)	13
Conception de la solution	14
Choix des technologies	14
Architecture	15
Modèles de données	17
Modèles d'utilisation	18

Contexte et objectif

Description du projet

Dans le cadre de mon cursus académique, j'ai eu l'occasion de réaliser régulièrement des projets techniques dans différents domaines tels que le développement web et le jeu vidéo. Tous ces projets sont généralement stockés sur des plateformes comme GitHub, accompagnés d'une documentation plus ou moins structurée (README, notes personnelles, etc.).

Cependant, avec les années, je me suis rendu compte que revenir sur d'anciens projets était compliqué, principalement sur l'aspect technique. Régulièrement, je me posais les mêmes questions : *Quelles sont les commandes pour lancer le projet ? Sur quelles plateformes chaque partie est-elle hébergée ?*

Tout cela rend donc leur consultation, leur mise à jour et leur exploitation plus complexes.

De plus, les solutions classiques de présentation de projets, comme les portfolios en ligne, sont principalement orientées vers une présentation visuelle, sans réellement prendre en compte les aspects techniques des projets.

Objectif du projet

C'est pourquoi l'objectif de ce projet est de concevoir et développer une plateforme web personnelle permettant de :

- Centraliser l'ensemble de mes projets techniques,
- Structurer les informations associées (description, stack technique, exécution, état actuel, etc.),
- Mettre en place une distinction claire entre les données que je souhaite rendre publiques et celles que je souhaite conserver privées, tout en permettant de modifier dynamiquement leur visibilité,
- Offrir une vue d'ensemble de mes projets, notamment en permettant de connaître leur dernière mise à jour grâce à l'automatisation de la récupération d'informations depuis des sources externes, principalement GitHub,
- Proposer une interface claire et professionnelle destinée aux recruteurs potentiels.

Ce projet vise ainsi à répondre à un double besoin :

- Un outil de gestion technique personnel,
- Un portfolio dynamique et pertinent pour la présentation de mes compétences.

Analyse de l'existant

Aujourd'hui, plusieurs solutions permettent de créer et de présenter des portfolios en ligne. Ces outils sont principalement destinés à offrir une **présentation visuelle des projets** et des compétences, notamment pour les développeurs, designers ou créatifs.

Parmi les solutions les plus répandues, on retrouve des plateformes de création de sites web telles que Wix, Squarespace, Webflow ou encore Portfoliobox. Ces outils permettent de concevoir rapidement un portfolio sans nécessiter de compétences techniques avancées.

Wix

Wix est une plateforme de création de sites web basée sur un éditeur visuel *drag-and-drop*. Elle propose de nombreux templates permettant de créer rapidement un portfolio professionnel.

Avantages	Limites
• Facilité d'utilisation • Mise en place rapide • Large choix de templates • Accessible sans compétences techniques	• Peu adapté à des projets techniques complexes • Faible intégration avec des outils comme GitHub • Pas de gestion avancée des données (runbook, commandes, etc.) • Difficulté à personnaliser la logique métier

Squarespace

Squarespace est une solution tout-en-un proposant des designs modernes et une interface intuitive, orientée vers la création de sites esthétiques.

Avantages	Limites
• Design professionnel et soigné • Interface simple à utiliser • Hébergement intégré	• Peu flexible pour des besoins techniques spécifiques • Pas de gestion fine des données techniques • Fonctionnalités limitées pour l'automatisation • Pas de distinction avancée public/privé

Webflow

Webflow est un outil no-code plus avancé, permettant de créer des interfaces complexes avec un rendu proche du développement front-end.

Avantages	Limites
• Grande flexibilité visuelle • Possibilité de créer des interfaces avancées • Génération de code propre	• Complexité d'apprentissage plus élevée • Limité pour la gestion de logique métier complexe • Pas conçu pour intégrer des workflows techniques (CI/CD, runbook, etc.)

Portfoliobox

Portfoliobox est une plateforme spécialisée dans la création de portfolios, notamment pour les profils créatifs.

Avantages	Limites
• Création rapide de portfolio • Interface simple • Adapté aux contenus visuels	• Très limité pour les projets techniques • Absence d'automatisation • Peu de personnalisation fonctionnelle • Aucun support pour des données techniques avancées

Synthèse

Comme indiqué précédemment, les solutions existantes que j'ai pu analyser permettent de manière générale toutes de créer rapidement des portfolios visuellement attractifs.

Cependant en analysant mon besoin actuel, ils présentent plusieurs limites importantes d'un point de vue techniques comme l'impossibilité d'intégrer des processus d'automatisation, le manque de contrôle sur la visibilité des données. Ce qui renforce l'idée unique de mon projet qui s'accroît plus sur le côté technique qui n'est pas présent sur ces différentes plateformes.

Cahier des charges

Exigences fonctionnelles

Les exigences fonctionnelles décrivent l'ensemble des fonctionnalités que le système doit proposer afin de répondre aux besoins définis précédemment.

Elles permettent de formaliser le comportement attendu de l'application du point de vue utilisateur. Je vais donc définir ci-dessous une liste des différentes exigences fonctionnelles, avec une description ainsi qu'une liste des attendus.

Exigences fonctionnelles	Description	Exigences attendus
Authentification et accès	Le système doit permettre de sécuriser l'accès aux fonctionnalités privées de l'application.	- L'utilisateur doit pouvoir se connecter via un formulaire de connexion sécurisé. - Le système doit gérer un utilisateur unique (administrateur). - Les mots de passe doivent être stockés de manière sécurisée (hash). - Les routes privées (ex : /admin) doivent être protégées. - La préproduction doit être accessible uniquement après authentification. - Le système doit gérer les sessions utilisateur.
Gestion des projets	Le système doit permettre à l'administrateur de gérer ses projets.	- L'utilisateur doit pouvoir créer un projet. - L'utilisateur doit pouvoir modifier un projet existant. - L'utilisateur doit pouvoir supprimer un projet. - L'utilisateur doit pouvoir archiver un projet.
Gestion de la visibilité	Le système doit permettre de contrôler précisément	- Un projet doit pouvoir être défini

	les informations visibles publiquement.	comme public ou privé. - Le système doit permettre de configurer la visibilité de certaines sections (stack technique, runbook, informations GitHub, statistiques) - Les informations privées ne doivent jamais être visibles côté public. - Les modifications de visibilité doivent être appliquées en temps réel.
Gestion du runbook	Le système doit permettre de documenter l'exécution des projets.	- L'utilisateur doit pouvoir définir plusieurs environnements (ex : local, docker). - Chaque environnement doit contenir plusieurs étapes ordonnées. - Le système doit permettre de copier facilement une commande. - Le runbook doit être affiché uniquement si autorisé.
Synchronisation GitHub	Le système doit intégrer des données dynamiques depuis GitHub.	- Le système doit récupérer la dernière activité du projet. - Le système doit afficher l'état de la branche principale. - Le système doit pouvoir afficher l'état de la préproduction. - Un mode de synchronisation automatique (CRON) doit être disponible.

		- L'utilisateur doit pouvoir déclencher une synchronisation manuelle. - Il doit être possible d'activer/désactiver la synchronisation automatique par projet.
Portfolio public	Le système doit proposer une interface publique de consultation des projets.	- Le système doit afficher une liste des projets publics. - L'utilisateur doit pouvoir filtrer les projets - Une page de détail doit être disponible pour chaque projet. - L'affichage doit respecter les règles de visibilité définies. - L'interface doit être responsive.
Statistiques	Le système doit fournir une vue globale des projets.	- Afficher le nombre total de projets - Afficher le nombre de projets actifs - Afficher le nombre de projets publics
Logs et traçabilité	Le système doit assurer un suivi des actions importantes.	- Le système doit enregistrer les connexions administrateur. - Le système doit enregistrer certaines actions (modification de projet, changement de visibilité) - Les logs doivent être consultables.
Monitoring	Le système doit assurer le suivi technique de l'application et collecter des informations sur son utilisation.	- Le système doit enregistrer les événements importants (consultation de projets, erreurs, actions utilisateur).

		- Le système doit permettre de collecter des données de navigation (pages consultées, projets les plus vus). - Le système doit envoyer ces données vers un outil de monitoring externe. - Le système doit permettre la visualisation des données sous forme de graphiques. - Le système doit permettre de consulter les statistiques d'utilisation (nombre de visites, fréquence, etc.).
Gestion des environnements	Le système doit distinguer plusieurs environnements.	- Le système doit gérer un environnement de production et pré-production - Les données doivent être isolées entre les environnements. - Le déploiement doit pouvoir être effectué indépendamment.
Déploiement et CI/CD	Le système doit être facilement déployable.	- L'application doit être conteneurisée avec Docker. - Le système doit être déployable via Docker Compose. - Une pipeline CI/CD doit être mise en place. - Les tests doivent être exécutés automatiquement avant déploiement. - Le déploiement doit pouvoir être automatisé via SSH.

		- Le déploiement lors d'une mise à jour doit se dérouler de manière à ce que l'utilisateur ne soit pas impacté pendant son utilisation à l'aide d'un déploiement en zero down time
--	--	--

Exigences non fonctionnelles

Les exigences non fonctionnelles définissent les contraintes techniques et qualitatives que doit respecter le système. Elles ne décrivent pas les fonctionnalités, mais les conditions dans lesquelles celles-ci doivent fonctionner (performance, sécurité, maintenabilité, etc.). Je vais donc définir ci-dessous une liste des différentes exigences fonctionnelles, avec une description ainsi qu'une liste des attendus.

1. Sécurité

Le système doit garantir la protection des données et des accès.

- Les mots de passe doivent être stockés sous forme hashée.
- Les routes sensibles doivent être protégées par authentification.
- Les accès non autorisés doivent être bloqués.
- Un système de rate limiting doit être appliqué sur les routes critiques (login).
- Les données privées ne doivent jamais être exposées côté public.
- Les communications doivent être sécurisées (HTTPS en production).
- Les sessions doivent être sécurisées (cookies sécurisés, expiration).

2. Performance

Le système doit être rapide et fluide pour l'utilisateur.

- Le temps de chargement des pages doit être optimisé.
- Les requêtes à la base de données doivent être efficaces.
- Les données externes (GitHub) doivent être mises en cache si nécessaire.
- L'application doit supporter plusieurs requêtes simultanées sans ralentissement notable.
- Les ressources statiques doivent être optimisées (images, scripts).

3. Maintenabilité

Le système doit être facilement modifiable et évolutif.

- Le code doit être structuré et organisé (architecture claire).
- Le projet doit utiliser TypeScript avec typage strict.
- Les bonnes pratiques de développement doivent être respectées.
- Les fonctions doivent être modulaires et réutilisables.
- La documentation technique doit être fournie.
- Les conventions de code (linting, formatage) doivent être respectées.

4. Testabilité

Le système doit pouvoir être testé efficacement.

- Des tests unitaires doivent être mis en place.
- Des tests de composants doivent être réalisés.
- Des tests end-to-end doivent être implémentés.
- Les tests doivent être exécutés automatiquement via CI/CD.
- Les fonctionnalités critiques doivent être couvertes par des tests.

Méthodologie & Organisation du travail

Afin d'assurer une gestion efficace du projet et de respecter les délais imposés, une organisation structurée a été mise en place. Le projet est réalisé selon une approche inspirée des méthodes Agile, adaptée à un contexte de développement individuel.

Méthodologie

Ce projet se base donc sur une méthodologie agile simplifiée, combinée à tableau Kanban. Le projet sera découpé en sprint hebdomadaires, chacun avec un objectif clair, un ensemble de tâches à faire (tickets) et des critères d'acceptation. A la fin de chaque sprint un bilan est réalisé dans lequel je validerai les tickets réalisés et les éventuels blocages ce qui me permettra d'ajuster le planning si nécessaire. Ensuite pour la création et gestion des tickets, tout se passe via un Kanban, chaque ticket contiendra une description, des critères d'acceptation et un niveau de priorité. Les tickets seront rangés selon le schéma suivant :



Organisation du travail

L'organisation du travail comprend la manière dont je vais mettre en place le projet, ainsi les bonnes conventions et habitudes à prendre pour garantir une bonne fluidité dans mon travail. En tout premier pour assurer un système de versionning et d'environnement de production/pré-production, le projet sera hébergé sur GitHub avec la stratégie suivante :

Plusieurs branches seront mises en place :

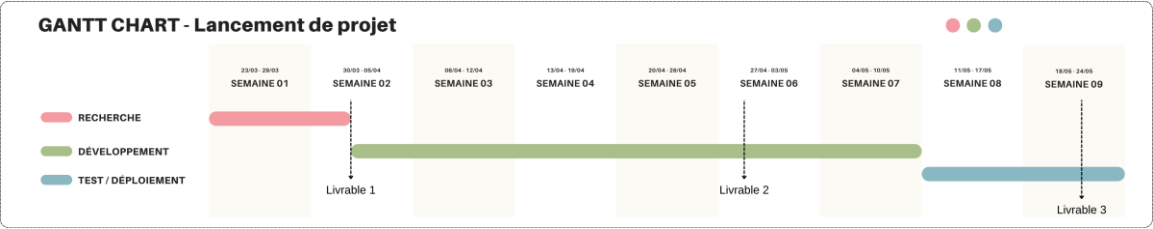
- Main : version stable (production)
- Develop : version en cours de développement (pré-production)
- Feature : dédiées à chaque nouvelle fonctionnalité
- Hotfix : correction urgente sur l'environnement de production

Chaque fonctionnalité est développée dans une branche dédiée, puis intégrée via une Pull Request.

Ensuite pour assurer la qualité du code je vais suivre plusieurs conventions, tels que conventionnels commits pour structurer les messages de commit. Ainsi que le respect des règles de linting et formatage. Au-delà des conventions à suivre, je vais mettre en place une CI/CD avec GitHub Actions sur chaque pull request pour permettre une vérification supplémentaire sur la qualité du code, exécuter les tests automatiquement et ainsi éviter la régression. Puis un déploiement automatisé de la branche develop vers la pré-production et de la branche main vers la production.

Pour suivre le rythme des sprints hebdomadaires mon travail sera organisé en plusieurs sessions de développement régulières dans la semaine.

Planning (Gantt)



Conception de la solution

Cette partie décrit les choix techniques et l'architecture du système mis en place pour répondre aux besoins définis précédemment. Elle vise à justifier les décisions prises en termes de technologies, d'organisation du système et de modélisation des données.

Choix des technologies

Les technologies utilisées ont été sélectionnées en fonction de leur modernité, de leur pertinence vis-à-vis des besoins du projet, ainsi que de mes expériences personnelles et des recherches effectuées.

Pour la gestion de la partie frontend et backend, le framework Next.js a été choisi. Ce framework fullstack permet de centraliser le développement au sein d'une seule application tout en offrant de bonnes performances et une structure claire. Il est utilisé en combinaison avec TypeScript, afin de garantir un typage strict, améliorer la maintenabilité du code et réduire les erreurs.

L'authentification est assurée par NextAuth, une solution intégrée à Next.js permettant de gérer de manière sécurisée les sessions utilisateur et les accès aux parties privées de l'application, tout en évitant de réimplémenter un système d'authentification complexe.

Pour la gestion des données, le choix s'est porté sur PostgreSQL, une base de données relationnelle robuste et adaptée à la structuration des informations du projet. Elle est associée à l'ORM Prisma, qui facilite les interactions avec la base de données grâce à un typage fort et une abstraction des requêtes SQL, permettant ainsi d'améliorer la productivité et la fiabilité du code.

Afin de garantir un environnement reproductible et faciliter le déploiement, l'application est conteneurisée à l'aide de Docker, avec une orchestration via Docker Compose. Cette approche permet d'assurer la cohérence entre les différents environnements (développement, préproduction, production) et de simplifier la gestion des dépendances.

Pour l'automatisation des tests et du déploiement, l'outil GitHub Actions est utilisé. Il permet de mettre en place une pipeline d'intégration continue (CI/CD), assurant la validation du code, l'exécution des tests et le déploiement automatisé de l'application.

Enfin, Nginx est utilisé en tant que reverse proxy. Il permet de gérer les accès aux différents services, de rediriger les requêtes vers les bons conteneurs et de sécuriser l'exposition de l'application via le nom de domaine, notamment dans le cadre de l'utilisation de sous-domaines (production, administration, préproduction).

Architecture

Comme présenté sur le schéma ci-dessous, l'architecture de l'application repose sur une séparation claire entre le front-end, le back-end et les services annexes.

Le front-end est développé en React (via Next.js) et constitue l'interface utilisateur. Il communique avec un back-end en Node.js, chargé de la gestion de la logique métier et des échanges avec les services externes.

Le back-end est directement connecté à une base de données PostgreSQL, utilisée pour stocker l'ensemble des données métier de l'application (projets, utilisateurs, configurations, etc.).

Dans une optique d'observabilité, une API interne est mise en place afin de collecter des événements (consultation de projets, erreurs, actions administrateur, etc.). Ces données sont ensuite envoyées vers une stack de monitoring composée de :

- Elasticsearch, qui permet de stocker et indexer les données,
- Grafana, utilisé pour visualiser ces données sous forme de tableaux de bord et de graphiques.

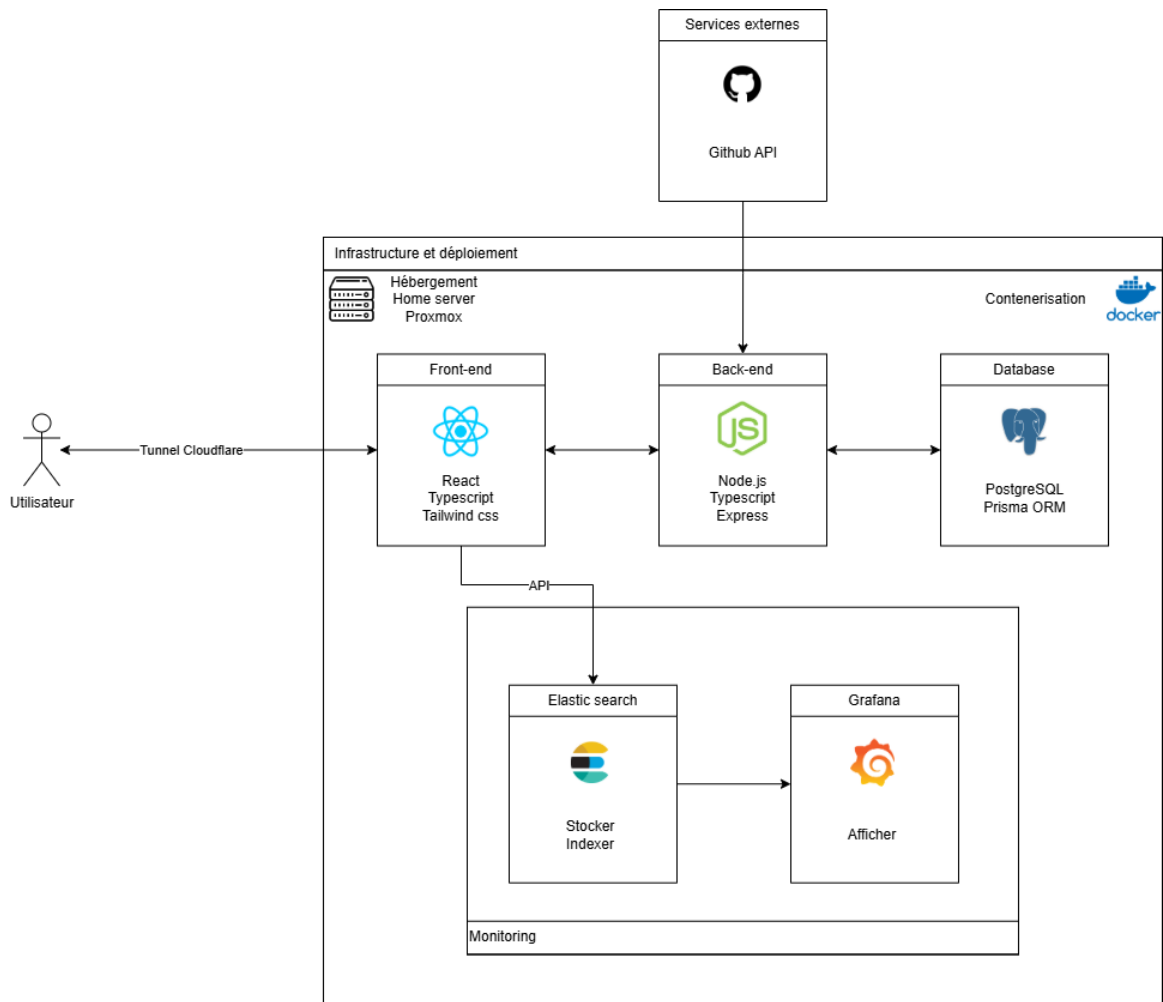
L'ensemble des composants de l'application (front-end, back-end, base de données, outils de monitoring) est conteneurisé à l'aide de Docker, afin de garantir un environnement reproductible et de faciliter le déploiement.

Le projet intègre également une pipeline CI/CD via GitHub Actions :

- la CI (Continuous Integration) permet d'exécuter automatiquement les tests et les vérifications (lint, build),
- la CD (Continuous Deployment) permet de déployer automatiquement l'application.

L'hébergement est assuré sur un serveur personnel fonctionnant sous Proxmox VE. L'application est accessible depuis l'extérieur via un nom de domaine, en passant par un tunnel sécurisé fourni par Cloudflare.

Enfin, afin d'enrichir les informations liées aux projets, l'application exploite l'API GitHub pour récupérer automatiquement certaines données (dernière mise à jour, informations du dépôt, etc.).



Modèles de données

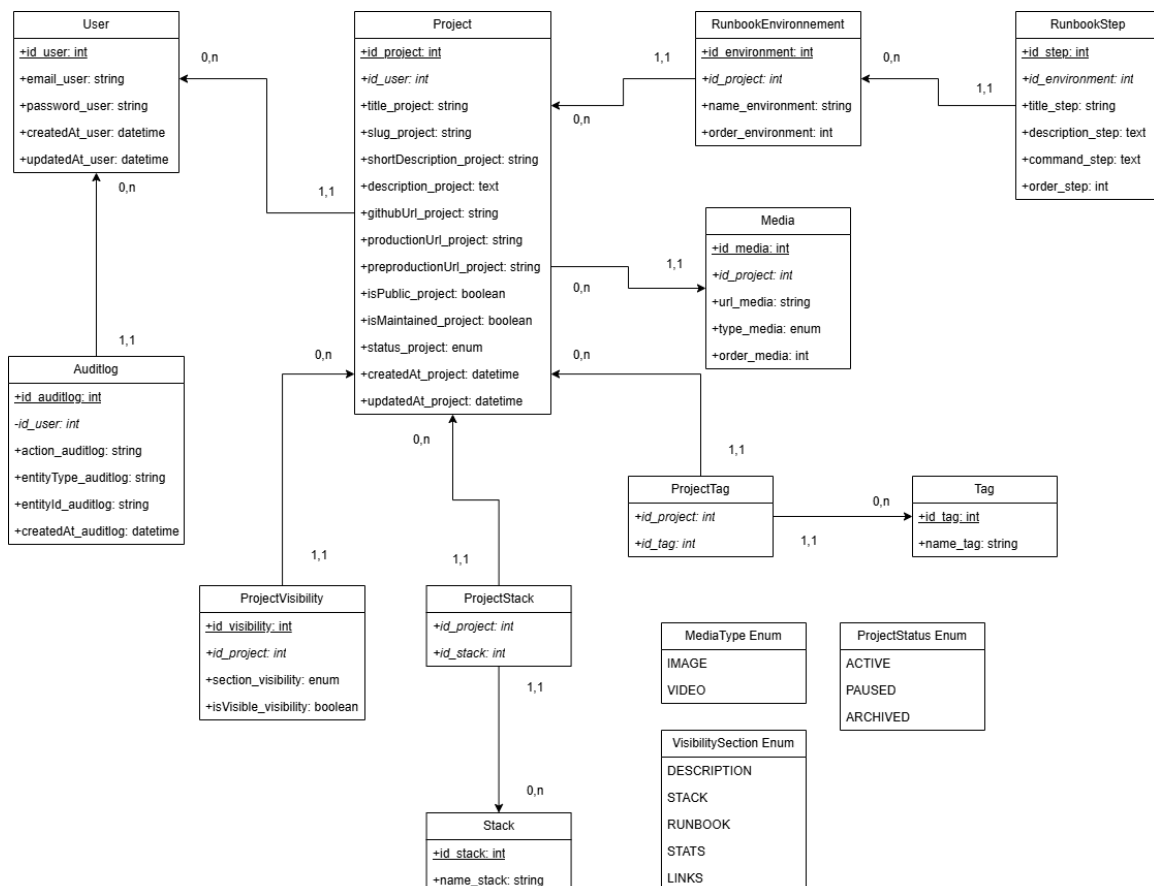
Le modèle de données de l'application est centré autour de l'entité Project, qui représente les projets techniques. Chaque projet est associé à un utilisateur (User) et contient des informations descriptives, des liens et des métadonnées (statut, visibilité, maintenance).

La classification des projets repose sur deux éléments : les tags (Tag) et les technologies (Stack), liés aux projets via des relations plusieurs-à-plusieurs (ProjectTag et ProjectStack).

Un système de runbook permet de documenter l'exécution des projets, avec des environnements (RunbookEnvironment) contenant des étapes (RunbookStep). Les médias (Media) permettent d'illustrer les projets.

La visibilité des différentes sections d'un projet est gérée via ProjectVisibility, offrant une distinction entre données publiques et privées. Enfin, les actions importantes sont tracées dans AuditLog.

Des types énumérés (enums) sont utilisés pour garantir la cohérence des données, notamment pour le statut des projets, les types de médias et les sections de visibilité.



Modèles d'utilisation

Le modèle d'utilisation présente les interactions entre les différents acteurs et le système. Il met en évidence les principales fonctionnalités accessibles aux visiteurs/recruteurs ainsi que celles réservées à l'administrateur.

